ST. VINCENT PALLOTTI COLLEGE OF ENGINEERING & TECHNOLOGY

(Affiliated to University of Nagpur)

DEPARTMENT OF INFORMATION TECHNOLOGY

SESSION: 2026-27

SEMESTER: V

SUBJECT: DATABASE MANAGEMENT SYSTEMS (24IT501T)

CASE STUDY ON

ONLINE EXAMINATION MANAGEMENT SYSTEM

Submitted by: Aditya Shahu

UID: 24010024

Table of Contents

1. Introduction
2. Objectives
3. Description and Problem Statement
4. Table Description
5. Relational Database Schema
5.1 SQL: Database and Table Creation
6. E-R Diagram
7. Sample Data
8. SQL Operations and Expected Output
8.1 Display All Exam Centers
8.2 Display All Registered Exams with Department and Subject
8.3 Display the Examination Schedule
8.4 Find Exams Scheduled on a Particular Date
8.5 Count Exams by Subject
8.6 Find Exam Centers with Highest Number of Scheduled Exams
8.7 Find Upcoming Scheduled Exams
8.8 Find Departments That Have Submitted Multiple Exams
8.9 Search for Exams by Title
8.10 Demonstrate a Subquery
8.11 Demonstrate Aggregate Functions
8.12 Create a Database View
9. Front-End Implementation and Database Connection
9.1 Tools Required
9.2 Frontend Screenshots
9.3 Backend Screenshots
10. Conclusion

1. Introduction

An Online Examination Management System is a specialized enterprise information system designed to manage the complex operations of academic institutions, autonomous universities, and competitive testing boards. Academic departments, faculties, course coordinators, and examination cell officers must coordinate the scheduling of numerous theoretical, practical, and objective examinations across multiple physical computer laboratories and virtual examination halls, each with its own candidate cohort, disciplinary domain, and designated time slot. Without a centralized database system, managing exam schedules, department details, center capacities, duration constraints, user credentials, and timetable modifications becomes highly error-prone, insecure, and administratively inefficient.

Traditional methods of maintaining examination schedules rely on manual ledgers, paper files, spreadsheet trackers, or disconnected departmental software. Such fragmented approaches lead to duplicate examination entries, conflicting time slots, examination hall double-booking, and difficulty in tracking which examination takes place in which center at any given time. Furthermore, unauthorized schedule tampering, absence of automated conflict prevention, and lack of comprehensive audit trails compromise the academic integrity and security of institutional examination operations.

The proposed Online Examination Management System addresses these critical challenges by implementing a normalized relational database using PostgreSQL as the backend and Python Flask as the web application framework. The front-end is developed using responsive HTML, CSS, and vanilla JavaScript, rendered through Flask's Jinja2 templating engine. The system delivers a centralized platform where academic departments can register, faculty members can submit examination entries with complete academic metadata and duration, and the examination controller can schedule and publish examinations in specific testing centers and time slots. Administrators oversee institutional operations through a protected, role-based portal.

PostgreSQL was selected for its robust support for relational integrity, advanced constraint enforcement (specifically btree_gist extension exclusion constraints to eliminate examination room double-booking), ACID transaction compliance, and high concurrency. Flask was chosen for its lightweight architecture, rapid development capabilities, seamless PostgreSQL integration via psycopg2, parameterized query execution to eliminate SQL injection vulnerabilities, and ideal fit for academic demonstration of full-stack database application development.

This case study presents a complete end-to-end implementation of the system, including the normalized relational database schema, entity-relationship design, realistic sample data, advanced SQL operations, Flask web backend, front-end user interfaces, and validation procedures.

2. Objectives

- **Design a normalized relational database** for academic examination operations covering users, academic departments, examination centers, subjects, exams, and schedules.
- **Maintain department, center, exam, schedule, and user records** with complete referential integrity and strict foreign key relationships.
- **Allow authorized users to register and log in** using industry-standard secure password hashing (Werkzeug scrypt).

- **Allow departments or authorized faculty to submit exams** for particular testing centers, dates, and dedicated time slots.
- **Prevent invalid schedules and conflicting time slots** in the same examination center using PostgreSQL btree_gist exclusion constraints.
- **Use primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, and DEFAULT** constraints to guarantee database consistency and prevent erroneous data insertion.
- **Demonstrate INSERT, SELECT, UPDATE, DELETE, JOIN,** filtering, aggregation, subqueries, database views, and atomic transactions.
- **Connect a Flask web application to PostgreSQL** using psycopg2 with parameterized SQL queries to eliminate SQL injection risks.
- **Provide an administrator dashboard** to review, verify, and approve submitted examination schedules with role-based access control.
- **Test database integrity and application functionality** through systematic query execution, real-world edge cases, and automated validation procedures.

3. Description and Problem Statement

3.1 Working Scenario

The Online Examination Management System supports academic departments and university testing cells in planning, approving, and conducting semester and internal examinations. The system manages the entire lifecycle of an examination, starting from examination creation by a departmental faculty coordinator, followed by examination cell verification, testing center allocation, and eventual scheduling on a designated date.

An exam in this system has a unique title, syllabus/instructions description, academic subject discipline, duration in minutes, and an associated department. An examination center represents a physical computer laboratory, proctored testing hall, or virtual proctoring cluster, characterized by center name, center type, capacity description, and active operational status. A schedule entry links a specific exam to an examination center and specifies the examination date, start time, end time, and status ('Scheduled', 'Completed', 'Cancelled', or 'Live').

Registered faculty and examiners can log in and submit examination proposals via web forms. Administrators and examination controllers can view all registered users, departments, examination centers, subjects, and scheduled sessions through an administrative portal. The system enforces role-based access control so that only authorized administrators can modify schedules, approve submissions, or change center allocations.

3.2 Problem Statement

The primary challenges that the system resolves are:

- **Duplicate or incomplete examination entries** leading to conflicting student timetables and faculty confusion.
- **Incorrect timing calculations** that cause examinations to overlap or leave critical gaps between test shifts.

- **Overlapping schedules in the same examination center,** resulting in room double-booking and infrastructure shortages.
- **Difficulty identifying which exam is active** at a particular time slot in a given campus laboratory or virtual cluster.
- **Unauthorized modifications to published timetables** compromising administrative security and examination secrecy.
- **Cumbersome manual procedures** for rescheduling, cancelling, or re-allocating examinations in emergency scenarios.
- **Lack of centralized scheduling data** across multiple engineering and science departments within the college.

3.3 Assumptions and Business Rules

- **Each exam belongs to exactly one department** and one academic subject discipline.
- **Each exam has exactly one creator** (faculty or coordinator) who submits it.
- **One examination center can host multiple scheduled examinations** on the same date across different time slots.
- **One exam can appear in multiple schedule records** over time (e.g., supplementary shifts or practical batches).
- **Each schedule record must reference** one valid exam and one active examination center.
- **An examination schedule's end time must be strictly later** than its start time (enforced via CHECK constraint).
- **Overlapping schedules in the same examination center** on the same date are prohibited and prevented at the database engine level.
- **User roles are strictly partitioned** between 'Admin' (Examination Cell) and 'User' (Faculty/Student).
- **Email addresses are unique** across all users and academic departments.
- **Password hashes are stored in the database;** plaintext passwords are never persisted.
- **The system operates on realistic demonstration data** modeled after college academic calendars.

4. Table Description

The database is designed with six normalized tables in Third Normal Form (3NF). Each table serves a specific entity domain and maintains strict referential integrity through primary and foreign key constraints.

Table Name	Purpose	Key Relationship / Reason
users	Stores user account information including administrators and faculty members.	One user can create multiple exams (one-to-many with exams).
departments	Stores academic department details such as department name, HOD/coordinator, contact email, and campus address.	One department can register multiple exams (one-to-many with exams).
exam_centers	Stores physical computer laboratories, testing centers, and virtual examination halls.	One center can host multiple scheduled exams across time slots (one-to-many with schedules).
subjects	Stores subject disciplines and course domains	One subject can be associated with multiple

	(e.g., DBMS, DSA, Operating Systems).	exams (one-to-many with exams).
exams	Stores examination master details including title, syllabus description, duration, and parent department.	Links to departments, subjects, and users via foreign keys. One exam can have multiple schedules.
schedules	Stores examination schedule records linking exams to centers with date, start time, end time, and status.	Links to exams and centers. Enforces exclusion constraints to prevent center double-booking.

Table 4.1: Database Table Descriptions

4.1 Relationships

- **DEPARTMENT to EXAM:** One-to-Many (one department registers multiple exams).
- **SUBJECT to EXAM:** One-to-Many (one subject discipline categorizes multiple exams).
- **USER to EXAM:** One-to-Many (one faculty member/user creates multiple exams).
- **EXAM to SCHEDULE:** One-to-Many (one exam can have multiple scheduled shifts or dates).
- **EXAM_CENTER to SCHEDULE:** One-to-Many (one examination center hosts multiple scheduled exams).

5. Relational Database Schema

The database is named online_examination_system_portal and resides in PostgreSQL's public schema. The following sections describe each table's structure, data types, constraints, and operational attributes.

5.1 Table: users

Column	Data Type	Constraints	Description
user_id	INTEGER	PRIMARY KEY, IDENTITY	Automatically generated unique user ID
full_name	VARCHAR(100)	NOT NULL	User's full academic name
email	VARCHAR(120)	NOT NULL, UNIQUE	User's official email address
password_hash	VARCHAR(255)	NOT NULL	Werkzeug scrypt password hash (never plaintext)
role	VARCHAR(10)	NOT NULL, DEFAULT 'User', CHECK	User role: 'Admin' or 'User'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	User account registration timestamp

5.2 Table: departments

Column	Data Type	Constraints	Description
department_id	INTEGER	PRIMARY KEY, IDENTITY	Automatically generated unique department ID
department_name	VARCHAR(150)	NOT NULL, UNIQUE	Academic department name

contact_person	VARCHAR(100)	NOT NULL	Department HOD or examination coordinator
email	VARCHAR(120)	NOT NULL, UNIQUE	Official department contact email
phone	VARCHAR(20)	NOT NULL	Department contact telephone number
address	TEXT	NOT NULL	Academic block and building office location
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Department registration timestamp

5.3 Table: exam_centers

Column	Data Type	Constraints	Description
center_id	INTEGER	PRIMARY KEY, IDENTITY	Automatically generated unique center ID
center_name	VARCHAR(100)	NOT NULL, UNIQUE	Examination center / computer lab name
center_type	VARCHAR(50)	NOT NULL, CHECK	Center type: 'Campus Lab', 'Virtual Hall', etc.
description	TEXT	NOT NULL	Center hardware, capacity, and seating description
active_status	BOOLEAN	DEFAULT TRUE	Whether the examination center is currently active

5.4 Table: subjects

Column	Data Type	Constraints	Description
subject_id	INTEGER	PRIMARY KEY, IDENTITY	Automatically generated unique subject ID
subject_name	VARCHAR(50)	NOT NULL, UNIQUE	Subject title (e.g., DBMS, DSA, Networks)
description	TEXT	NOT NULL	Academic syllabus and domain description

5.5 Table: exams

Column	Data Type	Constraints	Description
show_id / exam_id	INTEGER	PRIMARY KEY, IDENTITY	Automatically generated unique exam ID
title	VARCHAR(200)	NOT NULL	Official examination title

description	TEXT	NOT NULL	Examination instructions and syllabus coverage
department_id	INTEGER	NOT NULL, FK to departments	Academic department offering the exam
subject_id	INTEGER	NOT NULL, FK to subjects	Subject discipline categorization
duration_minutes	INTEGER	NOT NULL, CHECK > 0	Allocated test duration in minutes
created_by	INTEGER	NOT NULL, FK to users	Faculty user who created the exam record
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Exam creation timestamp

5.6 Table: schedules

Column	Data Type	Constraints	Description
schedule_id	INTEGER	PRIMARY KEY, IDENTITY	Automatically generated unique schedule ID
exam_id	INTEGER	NOT NULL, FK to exams	Scheduled examination reference
center_id	INTEGER	NOT NULL, FK to exam_centers	Allocated examination center / computer lab
exam_date	DATE	NOT NULL	Scheduled examination date
start_time	TIME	NOT NULL	Examination shift start time
end_time	TIME	NOT NULL, CHECK > start_time	Examination shift end time
status	VARCHAR(20)	NOT NULL, DEFAULT 'Scheduled', CHECK	Status: 'Scheduled', 'Completed', 'Cancelled', 'Live'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Schedule allocation timestamp

5.7 Schedule Conflict Prevention

To prevent overlapping examination schedules in the same testing center, PostgreSQL's btree_gist extension is employed to establish an exclusion constraint on the schedules table. The exclusion constraint ensures that no two schedule records referencing the same examination center and date possess overlapping time boundaries. If an operator attempts to book a center during an existing scheduled window, PostgreSQL automatically rejects the transaction with an exclusion violation error, guaranteeing absolute room allocation integrity.

5.8 SQL: Database and Table Creation

The following SQL statements create the database and all six normalized tables with constraints, indexes, and views. The database is initialized first, and subsequent DDL commands are executed within its context.

```sql
-- ======================================================
-- Online Examination Management System
-- Database: online_examination_system_portal
-- ======================================================

-- Step 1: Create database (run separately in psql/pgAdmin)
-- CREATE DATABASE online_examination_system_portal;

-- Step 2: Connect to database
-- \c online_examination_system_portal;

-- Step 3: Enable btree_gist for exclusion constraints
CREATE EXTENSION IF NOT EXISTS btree_gist;

-- ======================================================
-- TABLE 1: users
-- ======================================================
CREATE TABLE users (
    user_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(10) NOT NULL DEFAULT 'User'
        CHECK (role IN ('Admin', 'User')),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- ======================================================
-- TABLE 2: departments
-- ======================================================
CREATE TABLE departments (
    department_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    department_name VARCHAR(150) NOT NULL UNIQUE,
    contact_person VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    phone VARCHAR(20) NOT NULL,
    address TEXT NOT NULL,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- ======================================================
-- TABLE 3: exam_centers
-- ======================================================
CREATE TABLE exam_centers (
    center_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    center_name VARCHAR(100) NOT NULL UNIQUE,
    center_type VARCHAR(50) NOT NULL
        CHECK (center_type IN ('Campus Lab','Virtual Hall','Proctored Center',
                               'Department Lab','Cloud Server','Regional Center')),
    description TEXT NOT NULL,
    active_status BOOLEAN NOT NULL DEFAULT TRUE
);

-- ======================================================
-- TABLE 4: subjects
-- ======================================================
CREATE TABLE subjects (
    subject_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    subject_name VARCHAR(50) NOT NULL UNIQUE,
    description TEXT NOT NULL
);
```

```
-- ======================================================
-- TABLE 5: exams
-- ======================================================
CREATE TABLE exams (
    exam_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    description TEXT NOT NULL,
    department_id INTEGER NOT NULL REFERENCES departments(department_id)
        ON DELETE RESTRICT,
    subject_id INTEGER NOT NULL REFERENCES subjects(subject_id)
        ON DELETE RESTRICT,
    duration_minutes INTEGER NOT NULL CHECK (duration_minutes > 0),
    created_by INTEGER NOT NULL REFERENCES users(user_id)
        ON DELETE RESTRICT,
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- ======================================================
-- TABLE 6: schedules
-- ======================================================
CREATE TABLE schedules (
    schedule_id INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    exam_id INTEGER NOT NULL REFERENCES exams(exam_id)
        ON DELETE CASCADE,
    center_id INTEGER NOT NULL REFERENCES exam_centers(center_id)
        ON DELETE RESTRICT,
    exam_date DATE NOT NULL,
    start_time TIME NOT NULL,
    end_time TIME NOT NULL,
    status VARCHAR(20) NOT NULL DEFAULT 'Scheduled'
        CHECK (status IN ('Scheduled','Completed','Cancelled','Live')),
    created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT chk_time_order CHECK (end_time > start_time),
    CONSTRAINT no_overlap
        EXCLUDE USING gist (
            center_id WITH =,
            daterange(exam_date, exam_date, '[]') WITH &&
        )
        WHERE (status <> 'Cancelled')
);

-- ======================================================
-- INDEXES
-- ======================================================
CREATE INDEX idx_exams_department ON exams(department_id);
CREATE INDEX idx_exams_subject ON exams(subject_id);
CREATE INDEX idx_exams_creator ON exams(created_by);
CREATE INDEX idx_schedules_exam ON schedules(exam_id);
CREATE INDEX idx_schedules_center ON schedules(center_id);
CREATE INDEX idx_schedules_date ON schedules(exam_date);
CREATE INDEX idx_users_email ON users(email);

-- ======================================================
-- VIEW: exam_schedule_view
-- ======================================================
CREATE VIEW exam_schedule_view AS
SELECT
    s.schedule_id,
    e.title AS exam_title,
    sub.subject_name,
```

```
    d.department_name,
    c.center_name,
    c.center_type,
    s.exam_date,
    s.start_time,
    s.end_time,
    s.status,
    e.duration_minutes
FROM schedules s
JOIN exams e ON s.exam_id = e.exam_id
JOIN exam_centers c ON s.center_id = c.center_id
JOIN departments d ON e.department_id = d.department_id
JOIN subjects sub ON e.subject_id = sub.subject_id;
```

6. E-R Diagram

The following Entity-Relationship diagram represents the database schema using Mermaid erDiagram architecture. It illustrates all six core entities, primary keys (PK), unique keys (UK), foreign keys (FK), principal attributes, and relational cardinalities.

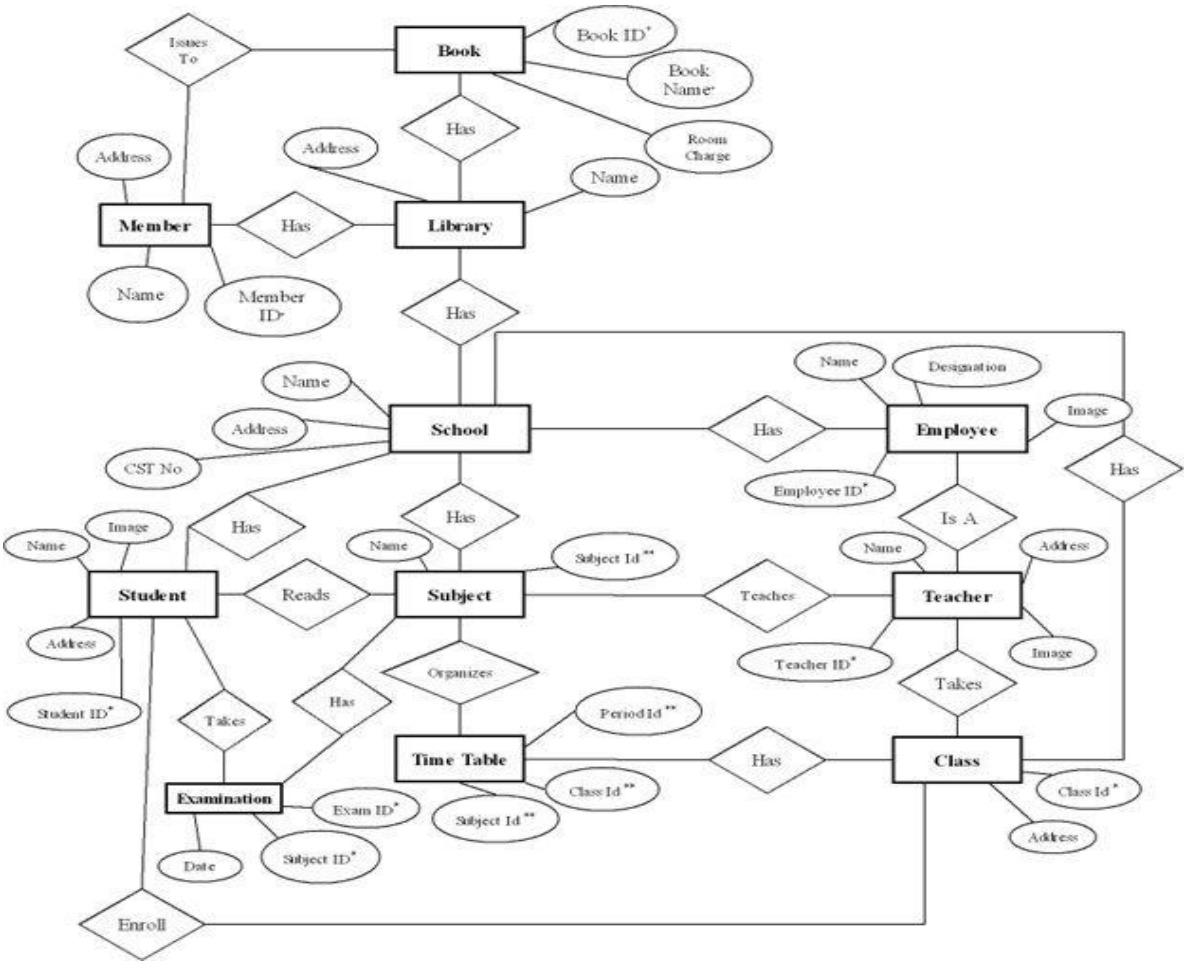


Figure 6.1: ER Diagram (Mermaid Syntax)

6.1 Cardinalities

- **USERS (1) -> (M) EXAMS:** One faculty user creates many exams.
- **DEPARTMENTS (1) -> (M) EXAMS:** One academic department registers many exams.
- **SUBJECTS (1) -> (M) EXAMS:** One subject discipline categorizes many exams.
- **EXAMS (1) -> (M) SCHEDULES:** One exam has many schedule entries across dates and shifts.
- **EXAM_CENTERS (1) -> (M) SCHEDULES:** One examination center hosts many scheduled exams.

7. Sample Data

The following seed dataset is provided for academic demonstration and system testing. It includes users, academic departments, examination centers, subjects, exams, and schedules with complete referential integrity. The demonstration password for all user accounts is Password@123 (hashed via Werkzeug scrypt).

7.1 Row Count Summary

Table	Rows	Description
users	20	Registered users including 2 exam administrators and 18 faculty members
departments	20	Academic engineering, sciences, and management departments
exam_centers	20	Campus computer laboratories, virtual testing halls, and regional centers
subjects	20	Academic subject disciplines and curriculum domains
exams	25	Examinations spanning mid-semester, unit tests, and end-semester tests
schedules	25	Scheduled examination shifts with varied operational statuses

Table 7.1: Sample Data Row Counts

7.2 Seed SQL Script

```
-- ======================================================
-- SEED DATA: online_examination_system_portal
-- Password for all demo accounts: Password@123
-- Hash generated by: werkzeug.security.generate_password_hash
-- ======================================================

-- USERS (20 rows) ------------------------------------------------------------
INSERT INTO users (full_name, email, password_hash, role) VALUES
('Admin User', 'admin@examportal.local',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc675023080bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'Admin'),
('Co Admin', 'coadmin@examportal.local',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc675023080bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'Admin'),
('Asha Patil', 'asha@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
```

```
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Rahul Deshmukh', 'rahul@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Priya Sharma', 'priya@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Amit Kumar', 'amit@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Sneha Joshi', 'sneha@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Vikram Singh', 'vikram@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Neha Gupta', 'neha@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Arjun Mehta', 'arjun@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Kavita Rao', 'kavita@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Sanjay Verma', 'sanjay@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Pooja Nair', 'pooja@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Rohan Das', 'rohan@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Meera Kulkarni', 'meera@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Aditya Rane', 'aditya@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Shreya Bose', 'shreya@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Karan Malhotra', 'karan@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc67502308
0bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
```

```
('Ananya Iyer', 'ananya@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc675023080bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User'),
('Devansh Patel', 'devansh@example.com',

'scrypt:32768:8:1$YXPaqcHh9gd3zjLM$3482f26fd1ba88687f18f09ba6e2020fcb91794b3b45f3cbc68f059cc675023080bc366ec82b4dd6af23b1b275da5be8aa27caaddb6a36cf4b80a47770b73dd2', 'User');
```

```
-- DEPARTMENTS (20 rows) ----------------------------------------------------
INSERT INTO departments (department_name, contact_person, email, phone, address) VALUES
('Department of Information Technology', 'Dr. Rajesh Kapoor', 'it@college.ac.in', '9876543210',
'Block A, Room 201'),
('Department of Computer Science', 'Dr. Sunita Agarwal', 'cse@college.ac.in', '9876543211',
'Block A, Room 301'),
('Department of AI and Data Science', 'Dr. K. Rajendran', 'aids@college.ac.in', '9876543212',
'Block B, Room 102'),
('Department of Electronics & TC', 'Dr. Prannoy Roy', 'etc@college.ac.in', '9876543213', 'Block
C, Room 205'),
('Department of Mechanical Engg', 'Dr. Nikhil Advani', 'mech@college.ac.in', '9876543214',
'Workshop Complex W1'),
('Department of Civil Engineering', 'Dr. Sudhanshu Vats', 'civil@college.ac.in', '9876543215',
'Block D, Room 101'),
('Department of Electrical Engg', 'Dr. Aroon Purie', 'ee@college.ac.in', '9876543216', 'Block C,
Room 108'),
('Department of Cyber Security', 'Dr. Deepak Chaurasia', 'cyber@college.ac.in', '9876543217',
'Block B, Room 305'),
('Department of Chemical Engg', 'Dr. Avinash Pandey', 'chem@college.ac.in', '9876543218',
'Science Wing, S204'),
('Department of Biotechnology', 'Dr. Arnab Goswami', 'biotech@college.ac.in', '9876543219', 'Life
Sciences Wing L1'),
('Department of Mathematics', 'Dr. Amit Bajaj', 'maths@college.ac.in', '9876543220', 'Science
Wing, S102'),
('Department of Physics', 'Dr. Megha Tata', 'physics@college.ac.in', '9876543221', 'Science Wing,
S105'),
('Department of Management Studies', 'Dr. Sylvia Bland', 'mba@college.ac.in', '9876543222', 'MBA
Block, M201'),
('Department of Computer Applications', 'Dr. Raj Nayak', 'mca@college.ac.in', '9876543223',
'Block A, Room 402'),
('Department of Humanities & Social Sc', 'Dr. Gautam Talwar', 'humanities@college.ac.in',
'9876543224', 'Main Admin Block, R12'),
('Department of Aerospace Engg', 'Dr. Punit Goenka', 'aero@college.ac.in', '9876543225',
'Aviation Annex, A101'),
('Department of Robotics & Automation', 'Dr. NP Singh', 'robotics@college.ac.in', '9876543226',
'Mechatronics Lab, M04'),
('Department of Environmental Engg', 'Dr. Warsi Khan', 'enviro@college.ac.in', '9876543227',
'Green Technology Wing'),
('Department of Mechatronics', 'Dr. Shalini Sethi', 'mechatronics@college.ac.in', '9876543228',
'Block C, Room 310'),
('Examination Regulatory Authority', 'Dr. S. Jaishankar', 'examcell@college.ac.in', '9876543229',
'Central Exam Cell, E01');
```

```
-- EXAM_CENTERS (20 rows) ---------------------------------------------------
INSERT INTO exam_centers (center_name, center_type, description, active_status) VALUES
('Turing Computer Center Lab 1', 'Campus Lab', 'High-performance lab with 60 workstations',
TRUE),
('Babbage Computing Lab 2', 'Campus Lab', 'Equipped with dual-monitor developer stations', TRUE),
```

```
('Ada Lovelace Virtual Hall A', 'Virtual Hall', 'Secure remote proctoring cluster with AI
monitoring', TRUE),
('Aryabhata Lab 104', 'Campus Lab', 'General engineering computer laboratory', TRUE),
('Von Neumann Center 201', 'Campus Lab', 'Advanced networking and database lab', TRUE),
('Ramanujan Testing Hall', 'Campus Lab', 'Specialized quantitative computing laboratory', TRUE),
('Shannon Proctored Cluster 1', 'Proctored Center', 'Biometric authenticated testing laboratory',
TRUE),
('Knuth Algorithm Testing Center', 'Campus Lab', 'Programming contests and practical
examinations', TRUE),
('Hopper Software Studio', 'Campus Lab', 'Dedicated software architecture and testing lab',
TRUE),
('Linus Open Source Lab', 'Campus Lab', 'Linux-based OS and systems programming lab', TRUE),
('Codd Relational Data Lab', 'Campus Lab', 'Optimized for high-throughput SQL exam environments',
TRUE),
('Dijkstra Distributed Systems Lab', 'Campus Lab', 'Network testing and distributed computing
facility', TRUE),
('Berners-Lee Web Innovation Lab', 'Campus Lab', 'Web technologies and frontend computing lab',
TRUE),
('Chomsky Language Lab', 'Campus Lab', 'Automata, compiler design and NLP test lab', TRUE),
('Gosling Java Testing Facility', 'Campus Lab', 'Object-oriented programming practical hall',
TRUE),
('Stroustrup C++ Computing Hall', 'Campus Lab', 'Embedded systems and algorithmic testing
cluster', TRUE),
('Ritchie Systems Lab', 'Campus Lab', 'Core operating systems and kernel lab', TRUE),
('Tesla Robotics Simulator Center', 'Campus Lab', 'Simulation cluster for CAD and robotics viva',
TRUE),
('Virtual Exam Hall Beta', 'Virtual Hall', 'Automated cloud-hosted browser exam portal', TRUE),
('Central Auditorium Testing Hall', 'Regional Center', 'Capacity for 300 candidates with CCTV
monitoring', TRUE);
```

```
-- SUBJECTS (20 rows) ------------------------------------------------------
INSERT INTO subjects (subject_name, description) VALUES
('Database Management Systems', 'Relational models, SQL, Normalization, ACID and indexing'),
('Data Structures & Algorithms', 'Stacks, Queues, Trees, Graphs, Sorting and Dynamic
Programming'),
('Operating Systems', 'Process synchronization, memory management, threads and file systems'),
('Computer Networks', 'OSI model, TCP/IP, routing algorithms and socket programming'),
('Software Engineering', 'Agile methodologies, SDLC, requirements and design patterns'),
('Machine Learning', 'Supervised, unsupervised learning, regression and neural networks'),
('Artificial Intelligence', 'Heuristic search, logic programming, planning and game theory'),
('Cyber Security & Cryptography', 'Symmetric/asymmetric crypto, hashing, network defense and
PKI'),
('Web Technologies', 'HTML5, CSS3, JavaScript, REST APIs and modern web frameworks'),
('Discrete Mathematics', 'Set theory, graph theory, combinatorics and predicate logic'),
('Cloud Computing', 'Virtualization, AWS/GCP architecture, Docker and microservices'),
('Compiler Design', 'Lexical analysis, syntax trees, parsing techniques and code generation'),
('Digital Signal Processing', 'Fourier transforms, digital filters, sampling and convolution'),
('Embedded Systems', 'Microcontrollers, ARM architecture, RTOS and IoT interfaces'),
('Computer Architecture', 'Instruction set architecture, pipelining, cache memory and ALU'),
('Data Mining & Warehousing', 'ETL processes, OLAP, association rule mining and data
clustering'),
('Blockchain Technology', 'Decentralized ledgers, consensus algorithms and smart contracts'),
('Internet of Things', 'Sensor networks, MQTT, edge computing and embedded hardware'),
('Mobile Application Development', 'Android, iOS, Flutter and reactive mobile UI architecture'),
('Human Computer Interaction', 'UI/UX heuristics, usability testing, accessibility and
prototyping');
```

```
-- EXAMS (25 rows) ------------------------------------------------------------
INSERT INTO exams (title, description, department_id, subject_id, duration_minutes, created_by)
VALUES
('DBMS Mid-Semester Examination', 'Mid-term evaluation covering relational schema, SQL and 3NF',
1, 1, 60, 3),
('Advanced DSA Practical Viva', 'Live algorithmic implementation and time complexity defense', 1,
2, 180, 4),
('Operating Systems Theory Paper', 'End-term written examination on scheduling and paging
algorithms', 5, 11, 60, 5),
('Computer Networks Unit Test 1', 'Assessment on OSI layers, subnetting and routing protocols',
6, 13, 45, 6),
('Software Engg Quiz & Review', 'Objective quiz on Agile frameworks, UML and testing metrics',
14, 4, 30, 7),
('Machine Learning Model Challenge', 'Hands-on evaluation building predictive models on live
datasets', 4, 1, 90, 8),
('AI Heuristic Search Term Paper', 'Written exam on A* algorithm, minimax and constraint
satisfaction', 12, 6, 60, 9),
('Cyber Security Defense Test', 'Network vulnerability assessment and cryptographic decoding',
16, 7, 45, 10),
('Web Technologies Code Viva', 'Development viva on responsive full-stack web applications', 15,
5, 60, 11),
('Discrete Math Mid-Term Test', 'Proof-based exam covering relations, graphs and recurrence', 17,
11, 60, 12),
('Cloud Systems Architecture Exam', 'Design paper on scalable microservices and Kubernetes
deployment', 16, 4, 30, 13),
('Compiler Design Syntax Analysis', 'Parsing tables, LL/LR parsers and intermediate code
generation', 15, 3, 30, 14),
('Digital Signal Processing Viva', 'Practical evaluation on FFT implementations and filter
design', 13, 18, 45, 15),
('Embedded Systems Lab Final', 'Firmware development on ARM Cortex microcontroller boards', 14,
16, 45, 16),
('Computer Architecture Viva', 'Defense on superscalar pipelines, branch prediction and caches',
14, 5, 90, 17),
('Data Mining Practical Exam', 'Evaluation on association mining and clustering in Python/R', 13,
6, 60, 18),
('Blockchain & Smart Contract Viva', 'Demonstration of Ethereum smart contracts and token
economics', 5, 7, 60, 19),
('IoT Sensor Networks Lab Test', 'Deployment of sensor telemetry over MQTT to cloud dashboards',
11, 6, 45, 20),
('Mobile App Flutter Unit Test', 'Interactive Flutter application coding and state management
exam', 4, 18, 30, 3),
('HCI Usability Audit Practical', 'Heuristic evaluation and accessibility auditing report
defense', 12, 17, 45, 4),
('Environmental Engineering Test', 'Assessment on pollution control norms and environmental
laws', 20, 19, 30, 5),
('Robotics Kinematics Exam', 'Mathematical analysis of forward and inverse robot kinematics', 1,
2, 60, 6),
('Applied Cryptography Final Exam', 'Advanced paper on zero-knowledge proofs and elliptic
curves', 15, 4, 45, 7),
('Decentralized Apps Project Viva', 'Capstone project evaluation on Web3 and smart contract
security', 17, 7, 120, 8),
('Departmental Aptitude Assessment', 'Comprehensive general engineering and analytical aptitude
test', 9, 20, 30, 9);
```

```
-- SCHEDULES (25 rows) ---------------------------------------------------------
INSERT INTO schedules (exam_id, center_id, exam_date, start_time, end_time, status) VALUES
( 1, 1, '2026-10-20', '06:00', '07:00', 'Scheduled'),
( 2, 5, '2026-10-20', '14:00', '17:00', 'Scheduled'),
( 3, 7, '2026-10-20', '20:00', '21:00', 'Scheduled'),
```

```
( 4, 9, '2026-10-20', '22:00', '22:45', 'Scheduled'),
( 5, 7, '2026-10-21', '20:30', '21:00', 'Scheduled'),
( 6, 2, '2026-10-21', '21:00', '22:30', 'Scheduled'),
( 7, 11, '2026-10-21', '19:00', '20:00', 'Scheduled'),
( 8, 16, '2026-10-21', '18:00', '18:45', 'Scheduled'),
( 9, 15, '2026-10-22', '20:00', '21:00', 'Scheduled'),
(10, 17, '2026-10-22', '21:00', '22:00', 'Scheduled'),
(11, 8, '2026-10-22', '20:30', '21:00', 'Scheduled'),
(12, 8, '2026-10-23', '20:30', '21:00', 'Scheduled'),
(13, 12, '2026-10-23', '19:00', '19:45', 'Scheduled'),
(14, 9, '2026-10-23', '20:00', '20:45', 'Scheduled'),
(15, 10, '2026-10-24', '21:00', '22:30', 'Scheduled'),
(16, 15, '2026-10-24', '18:00', '19:00', 'Scheduled'),
(17, 7, '2026-10-24', '20:00', '21:00', 'Scheduled'),
(18, 11, '2026-10-25', '20:00', '20:45', 'Scheduled'),
(19, 2, '2026-10-25', '18:00', '18:30', 'Scheduled'),
(20, 12, '2026-10-25', '17:00', '17:45', 'Scheduled'),
(21, 13, '2026-10-26', '06:00', '06:30', 'Scheduled'),
(22, 5, '2026-10-26', '16:00', '17:00', 'Scheduled'),
(23, 9, '2026-10-26', '22:00', '22:45', 'Cancelled'),
(24, 7, '2026-10-27', '20:00', '22:00', 'Scheduled'),
(25, 13, '2026-10-27', '18:00', '18:30', 'Scheduled');
```

8. SQL Operations and Expected Output

This section demonstrates key SQL operations with complete queries and representative expected outputs. All outputs are illustrative and correspond to the sample dataset provided in Section 7.

8.1 Display All Exam Centers

Purpose: List all registered examination centers with their IDs, names, types, and active status.

```
SELECT center_id, center_name, center_type, active_status
FROM exam_centers
ORDER BY center_id;
```

Expected Output (representative):

center_id	center_name	center_type	active_status
1	Turing Computer Center Lab 1	Campus Lab	true
2	Babbage Computing Lab 2	Campus Lab	true
3	Ada Lovelace Virtual Hall A	Virtual Hall	true
4	Aryabhata Lab 104	Campus Lab	true
5	Von Neumann Center 201	Campus Lab	true

The query retrieves all 20 examination centers ordered by center_id.

8.2 Display All Registered Exams with Department and Subject

Purpose: Use INNER JOIN to display exams with their associated academic department and subject names.

```
SELECT e.exam_id, e.title, d.department_name, sub.subject_name, e.duration_minutes
FROM exams e
INNER JOIN departments d ON e.department_id = d.department_id
INNER JOIN subjects sub ON e.subject_id = sub.subject_id
ORDER BY e.exam_id;
```

Expected Output (representative):

exam_id	title	department_name	subject_name	duration_minutes
1	DBMS Mid-Semester Examination	Department of Information Technology	Database Management Systems	60
2	Advanced DSA Practical Viva	Department of Information Technology	Data Structures & Algorithms	180
3	Operating Systems Theory Paper	Department of Mechanical Engg	Cloud Computing	60
4	Computer Networks Unit Test 1	Department of Civil Engineering	Digital Signal Processing	45
5	Software Engg Quiz & Review	Department of Computer Applications	Computer Networks	30

INNER JOIN ensures only exams with valid department and subject references are returned.

8.3 Display the Examination Schedule

Purpose: Join schedules, exams, and examination centers to display complete timetable information.

```
SELECT e.title AS exam_title, c.center_name, s.exam_date,
       s.start_time, s.end_time, s.status
FROM schedules s
JOIN exams e ON s.exam_id = e.exam_id
JOIN exam_centers c ON s.center_id = c.center_id
ORDER BY s.exam_date, s.start_time;
```

Expected Output (representative):

exam_title	center_name	exam_date	start_time	end_time	status
DBMS Mid-Semester Examination	Turing Computer Center Lab 1	2026-10-20	06:00	07:00	Scheduled
Advanced DSA Practical Viva	Von Neumann Center 201	2026-10-20	14:00	17:00	Scheduled
Operating Systems Theory Paper	Shannon Proctored Cluster 1	2026-10-20	20:00	21:00	Scheduled
Computer Networks Unit Test 1	Hopper Software Studio	2026-10-20	22:00	22:45	Scheduled

8.4 Find Exams Scheduled on a Particular Date

Purpose: Filter examination schedules using WHERE with a specific date.

```
SELECT e.title, c.center_name, s.start_time, s.end_time, s.status
FROM schedules s
JOIN exams e ON s.exam_id = e.exam_id
JOIN exam_centers c ON s.center_id = c.center_id
WHERE s.exam_date = '2026-10-20'
ORDER BY s.start_time;
```

Expected Output (representative):

title	center_name	start_time	end_time	status
DBMS Mid-Semester Examination	Turing Computer Center Lab 1	06:00	07:00	Scheduled
Advanced DSA Practical Viva	Von Neumann Center 201	14:00	17:00	Scheduled
Operating Systems Theory Paper	Shannon Proctored Cluster 1	20:00	21:00	Scheduled
Computer Networks Unit Test 1	Hopper Software Studio	22:00	22:45	Scheduled

8.5 Count Exams by Subject

Purpose: Use GROUP BY and COUNT to show the number of exams per academic subject discipline.

```
SELECT sub.subject_name, COUNT(e.exam_id) AS exam_count
FROM subjects sub
LEFT JOIN exams e ON sub.subject_id = e.subject_id
GROUP BY sub.subject_name
ORDER BY exam_count DESC;
```

Expected Output (representative):

subject_name	exam_count
Machine Learning	3
Database Management Systems	3
Computer Networks	2
Data Structures & Algorithms	2
Software Engineering	2

LEFT JOIN ensures subjects with no registered exams are included with a count of 0.

8.6 Find Exam Centers with Highest Number of Scheduled Exams

Purpose: Use aggregation and ordering to identify the busiest examination computer laboratories.

```
SELECT c.center_name, COUNT(s.schedule_id) AS schedule_count
FROM exam_centers c
JOIN schedules s ON c.center_id = s.center_id
GROUP BY c.center_name
ORDER BY schedule_count DESC
LIMIT 10;
```

Expected Output (representative):

center_name	schedule_count
Shannon Proctored Cluster 1	3
Hopper Software Studio	3
Knuth Algorithm Testing Center	2
Turing Computer Center Lab 1	2
Dijkstra Distributed Systems Lab	2

8.7 Find Upcoming Scheduled Exams

Purpose: Find examinations scheduled for today or later. Note: The output depends on the current date at runtime. The sample data uses October 2026 dates for academic calendar demonstration.

```
SELECT e.title, c.center_name, s.exam_date, s.start_time, s.status
FROM schedules s
JOIN exams e ON s.exam_id = e.exam_id
JOIN exam_centers c ON s.center_id = c.center_id
WHERE s.exam_date >= CURRENT_DATE
  AND s.status IN ('Scheduled', 'Live')
ORDER BY s.exam_date, s.start_time;
```

Note: CURRENT_DATE is evaluated at query execution time. For demonstration, replace with a fixed date such as '2026-10-20'.

8.8 Find Departments That Have Submitted Multiple Exams

Purpose: Use GROUP BY and HAVING to filter academic departments with more than one exam submission.

```
SELECT d.department_name, COUNT(e.exam_id) AS exam_count
FROM departments d
JOIN exams e ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(e.exam_id) > 1
ORDER BY exam_count DESC;
```

Expected Output (representative):

department_name	exam_count
Department of Information Technology	2
Department of Computer Applications	2
Department of Management Studies	2

HAVING filters groups after aggregation, unlike WHERE which filters individual rows.

8.9 Search for Exams by Title

Purpose: Search for examinations using a parameterized LIKE query.

```
-- Application parameterized query (Python/Flask):
-- cursor.execute("SELECT * FROM exams WHERE title ILIKE %s", ('%' + search + '%',))

SELECT e.exam_id, e.title, d.department_name, sub.subject_name
FROM exams e
JOIN departments d ON e.department_id = d.department_id
JOIN subjects sub ON e.subject_id = sub.subject_id
WHERE e.title ILIKE '%database%'
ORDER BY e.title;
```

ILIKE provides case-insensitive search. Parameterized queries prevent SQL injection attacks.

8.10 Demonstrate a Subquery

Purpose: Identify academic departments whose number of registered exams exceeds the average number of exams per department.

```
SELECT d.department_name, COUNT(e.exam_id) AS exam_count
FROM departments d
JOIN exams e ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
HAVING COUNT(e.exam_id) > (
    SELECT AVG(exam_count)
    FROM (
        SELECT COUNT(*) AS exam_count
        FROM exams
        GROUP BY department_id
    ) AS department_counts
);
```

The subquery computes the average number of exams per department, then the outer query filters departments exceeding that average.

8.11 Demonstrate Aggregate Functions

Purpose: Demonstrate COUNT, AVG, MIN, MAX, and SUM on meaningful numerical columns (duration_minutes).

```
SELECT
    COUNT(*) AS total_exams,
    AVG(duration_minutes) AS avg_duration,
    MIN(duration_minutes) AS min_duration,
    MAX(duration_minutes) AS max_duration,
    SUM(duration_minutes) AS total_duration
FROM exams;
```

Expected Output (representative):

total_exams	avg_duration	min_duration	max_duration	total_duration

25	61.20	30	180	1530

All aggregate functions operate on the numeric duration_minutes column.

8.12 Create a Database View

Purpose: Create a view that joins schedules, exams, centers, and departments for simplified querying.

```
CREATE VIEW exam_schedule_view AS
SELECT
    s.schedule_id,
    e.title AS exam_title,
    sub.subject_name,
    d.department_name,
    c.center_name,
    c.center_type,
    s.exam_date,
    s.start_time,
    s.end_time,
    s.status,
    e.duration_minutes
FROM schedules s
JOIN exams e ON s.exam_id = e.exam_id
JOIN exam_centers c ON s.center_id = c.center_id
JOIN departments d ON e.department_id = d.department_id
JOIN subjects sub ON e.subject_id = sub.subject_id;

SELECT * FROM exam_schedule_view
WHERE exam_date = '2026-10-20'
ORDER BY start_time;
```

The view encapsulates the complex four-table join logic for intuitive and secure frontend querying.

9. Front-End Implementation and Database Connection

9.1 Tools Required

Tool	Purpose
PostgreSQL	Relational database management system for storing all data with ACID guarantees.
pgAdmin 4	Graphical administration tool for managing PostgreSQL databases, queries, and constraints.
Python 3.10+	Programming language for the Flask web application backend and business logic.
Flask	Lightweight, modular Python web framework for routing, session handling, and Jinja2 templating.
psycopg2-binary	PostgreSQL database adapter for Python supporting parameterized queries and connection pooling.
Modern Web Browser	Client environment for accessing the responsive user, faculty, and administrative interfaces.

Table 9.1: Technology Stack and Tools

9.2 Frontend Screenshots

The ExamHub public landing portal provides an intuitive dashboard for accessing live timetables and scheduling examinations.

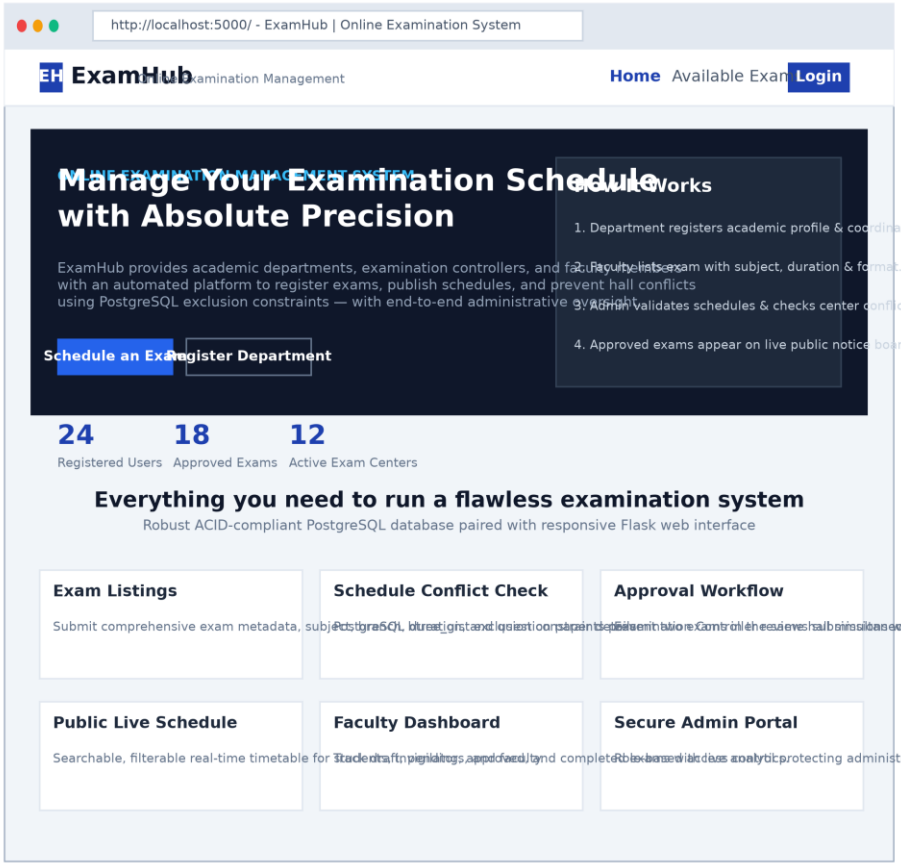


Figure 9.1: ExamHub Home & Portal Landing Page

The public examination schedule features responsive filters by subject, laboratory, and date, paired with an administrative KPI overview.

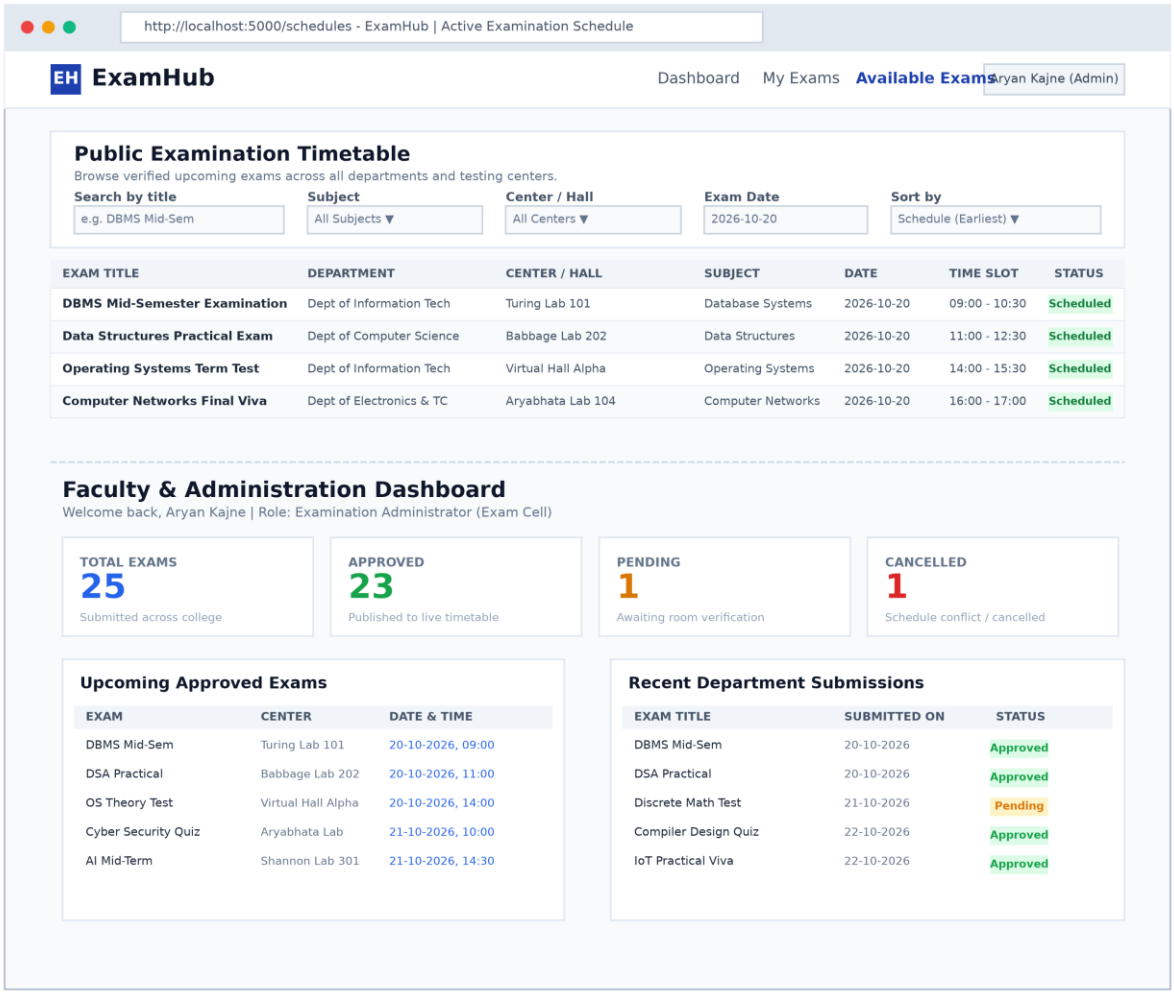


Figure 9.2: Public Examination Timetable & Faculty Dashboard

Faculty members can submit examination details through an authenticated form protected by PostgreSQL conflict validation.

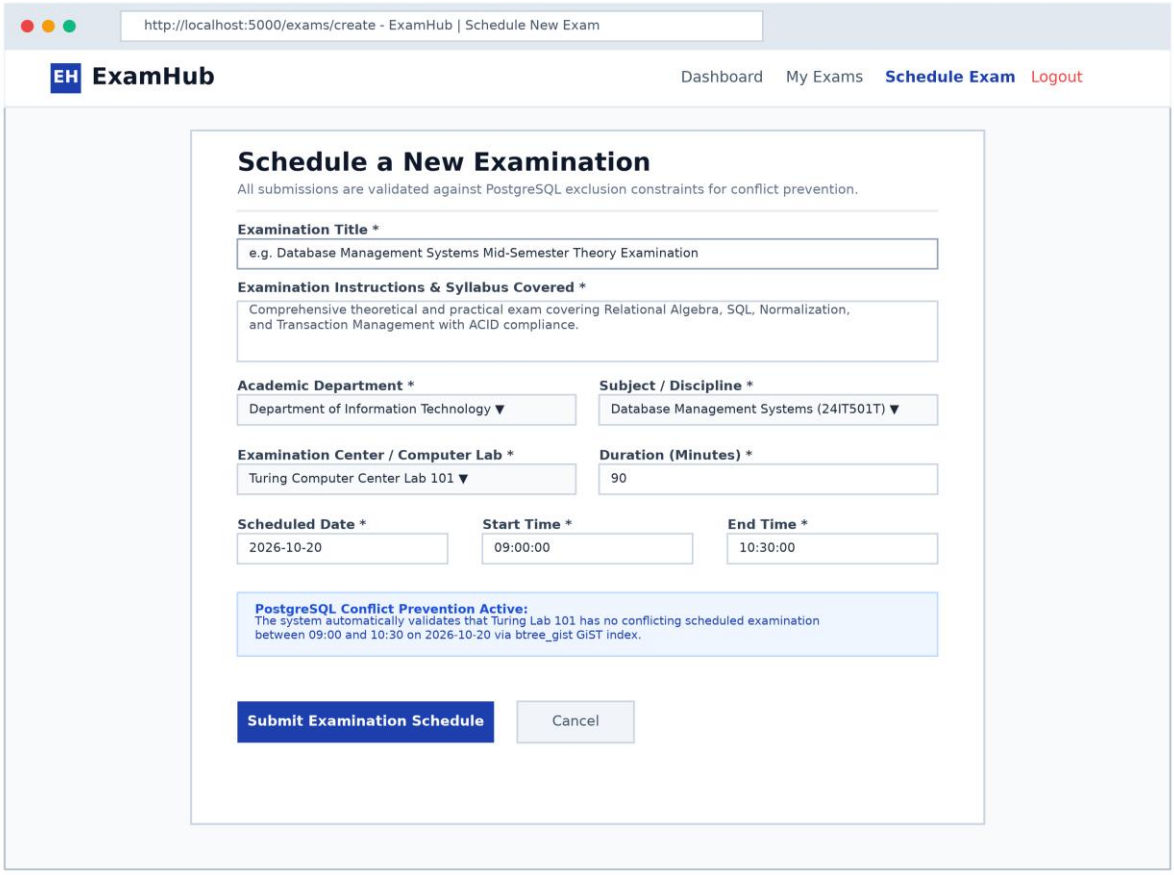


Figure 9.3: Schedule a New Examination Web Interface

9.3 Backend Screenshots

The backend infrastructure pairs a PostgreSQL database server with a Flask application using parameterized query execution.

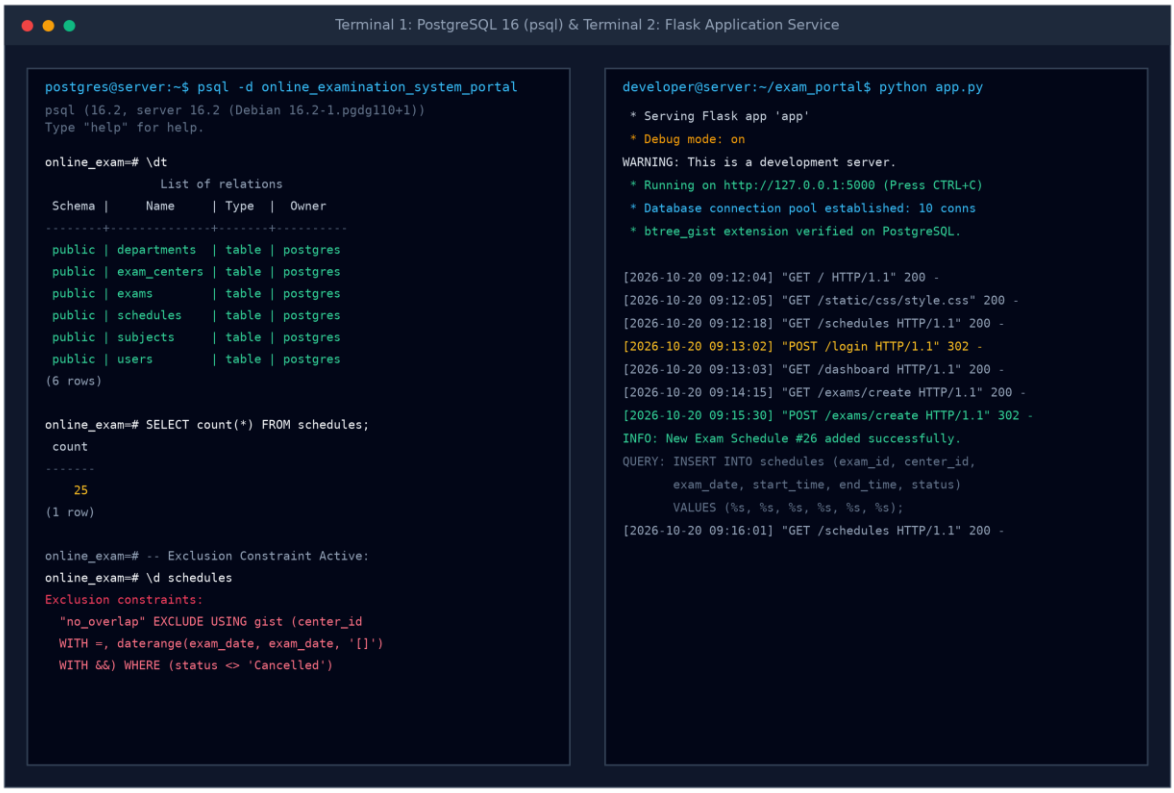


Figure 9.4: PostgreSQL psql Terminal and Flask Application Service

Below is the core Python Flask database connection and schedule creation controller illustrating parameterized queries and exclusion error handling:

```
from flask import Flask, request, render_template, redirect, url_for, flash
import psycopg2
from psycopg2 import pool
from werkzeug.security import check_password_hash, generate_password_hash

app = Flask(__name__)
app.secret_key = 'super_secret_exam_portal_key'

# PostgreSQL Connection Pool
db_pool = psycopg2.pool.SimpleConnectionPool(
    minconn=1, maxconn=20,
    dbname="online_examination_system_portal",
    user="postgres", password="password", host="localhost", port="5432"
)

@app.route('/exams/create', methods=['GET', 'POST'])
def create_schedule():
    if request.method == 'POST':
        exam_id = request.form['exam_id']
        center_id = request.form['center_id']
        exam_date = request.form['exam_date']
        start_time = request.form['start_time']
        end_time = request.form['end_time']

        conn = db_pool.getconn()
        try:
```

```
            with conn.cursor() as cur:
                # Parameterized query protects against SQL injection
                sql = """
                    INSERT INTO schedules (exam_id, center_id, exam_date, start_time, end_time,
status)
                    VALUES (%s, %s, %s, %s, %s, 'Scheduled')
                    RETURNING schedule_id;
                """
                cur.execute(sql, (exam_id, center_id, exam_date, start_time, end_time))
                new_id = cur.fetchone()[0]
                conn.commit()
                flash(f"Exam scheduled successfully! Schedule ID: {new_id}", "success")
                return redirect(url_for('public_schedule'))
        except psycopg2.errors.ExclusionViolation:
            conn.rollback()
            flash("CONFLICT DETECTED: This examination center is already booked for an
overlapping time slot!", "danger")
        except Exception as e:
            conn.rollback()
            flash(f"Database error occurred: {str(e)}", "danger")
        finally:
            db_pool.putconn(conn)

    return render_template('create_schedule.html')
```

10. Conclusion

The Online Examination Management System demonstrates the practical application of relational database design principles in an institutional academic testing scenario. The system successfully implements a normalized database schema with six related tables, ensuring data integrity through primary keys, foreign keys, UNIQUE, NOT NULL, CHECK, and DEFAULT constraints.

Key DBMS concepts demonstrated in this project include:

- **Normalization:** The database is designed in third normal form (3NF), eliminating update anomalies and data redundancy.
- **Referential Integrity:** Foreign key constraints with ON DELETE RESTRICT and CASCADE maintain relational consistency across all tables.
- **Constraint Enforcement:** CHECK constraints validate temporal order and values; UNIQUE constraints prevent duplicates.
- **Exclusion Constraints:** The btree_gist extension prevents overlapping examination schedules in the same testing center.
- **SQL Operations:** The system demonstrates INSERT, SELECT, UPDATE, DELETE, JOIN, GROUP BY, HAVING, subqueries, views, and transactions.
- **Application Integration:** Flask connects to PostgreSQL using psycopg2 with parameterized queries, preventing SQL injection.
- **Role-Based Access Control:** The system securely distinguishes between regular users (faculty/students) and administrators.

The centralized examination scheduling system eliminates problems associated with manual exam management, including duplicate entries, room double-booking, and difficulty tracking active test sessions across college departments.

Submitted To: Prof. Jayant S Rohankar

Course Faculty, Department of IT